Lab 4 – Data Processing II

Combine, summarize, reshape

Course text for the notebook `lab4_data_processing_II.py`

dr. Maarten Slembrouck

What is this lab about?

Last week you turned messy exports into tidy tables: one row per athlete, one row per test. But a coach's questions are rarely about one row. "How tall are our defenders on average?" is about a *group* of rows. "How fast do our goalkeepers sprint?" needs information from *two* tables. "Has everyone improved since the first test?" asks for the table to be *arranged* differently. And the heart-rate data of a training session is a *time series*, with one measurement per second.

After this lab you can summarize data per group with `groupby`, combine tables with `merge`, reshape data with `melt` and `pivot_table`, and compute basic features of a time series (heart-rate zones, rolling average, resampling).

Just like last week, every example was actually run on the club's data. Read this text next to the notebook and type the examples yourself.

1 Starting point: last week's cleaned data

The notebook starts with the cleaning from lab 3, in compact form. Two helper functions do the work: `clean_positions` turns the 13 spellings into 4 positions, `fix_sprint_units` converts milliseconds into seconds.

```
import pandas as pd

def clean_positions(position_column):
    position_map = {
        "forward": "Forward", "fwd": "Forward",
        "midfielder": "Midfielder", "mid": "Midfielder",
        "defender": "Defender", "def": "Defender",
        "goalkeeper": "Goalkeeper", "gk": "Goalkeeper",
    }
    return position_column.str.strip().str.lower().map(position_map)

def fix_sprint_units(seconds_column):
    return seconds_column.where(seconds_column < 20, seconds_column / 1000)

REFERENCE_DATE = pd.Timestamp("2026-09-14")

athletes = pd.read_csv(f"{DATA_DIR}/athletes.csv").assign(
    position=lambda d: clean_positions(d["position"]),
    weight_kg=lambda d: d["weight_kg"].str.replace(",", ".").astype(float),
    age=lambda d: (REFERENCE_DATE - pd.to_datetime(d["birthdate"])).dt.days // 365,
)
tests = pd.read_csv(f"{DATA_DIR}/tests.csv").assign(
    sprint_10m_s=lambda d: fix_sprint_units(d["sprint_10m_s"]),
    sprint_30m_s=lambda d: fix_sprint_units(d["sprint_30m_s"]),
)
```

Read this code calmly: you saw everything in it last week. Note that the duplicated athlete is

deliberately not removed yet. That becomes important in section 3.

Recap: reading error messages

The notebook starts with a broken cell that you have to repair. An error message looks scary, but the most important information is always on the **last line**: the kind of error and a short explanation. Two errors you will certainly meet today:

```
athletes["positio"]
```

output

```
KeyError: 'positio'
```

A `KeyError` means: this key (here: column name) does not exist. Almost always a typo.

```
pd.Series(["57,5", "60,3"]).astype(float)
```

output

```
ValueError: could not convert string to float: '57,5'
```

A `ValueError` from `astype(float)` means: I can't turn this text into a number. Here the comma is in the way.

2 Summarizing groups with groupby

2.1 Split, apply, combine

Suppose the coach wants the average age per position. In your head you do three things:

1. **split**: divide the athletes into four groups, one per position;
2. **apply**: compute the mean for each group;
3. **combine**: put the four means in one table.



In pandas that is one line. `groupby("position")` splits, `["age"]` picks the column, `.mean()` applies the mean and combines:

```
mean_age_per_position = athletes.groupby("position")["age"].mean().round(1)
mean_age_per_position
```

output

```
position
Defender      14.6
Forward       13.2
Goalkeeper    14.4
Midfielder    14.5
Name: age, dtype: float64
```

The result is a Series with the positions as its *index*. Instead of `.mean()` you can also use `.median()`, `.min()`, `.max()`, `.sum()` or `.count()`.

`size()` or `count()`?

`.size()` counts the rows per group. `.count()` counts the *filled-in* values in a column and skips `NaN`. To know how many athletes there are per position, use `.size()`.

```
athletes.groupby("position").size()
```

output

```
position
Defender      12
Forward       14
Goalkeeper     5
Midfielder    12
dtype: int64
```

2.2 Numbers into categories: `pd.cut`

Youth teams are divided into age groups: U13, U15, U17, U19. `pd.cut` divides a numeric column into bins. You give the edges (`bins`) and the names of the bins (`labels`):

```
athletes_with_age_group = athletes.assign(
    age_group=lambda d: pd.cut(
        d["age"], bins=[10, 12, 14, 16, 18], labels=["U13", "U15", "U17", "U19"]
    )
)
athletes_with_age_group[["name", "age", "age_group"]].head()
```

output

```
             name  age age_group
0  Yana De Backer   16       U17
1    Stan Mertens   14       U15
2    Amber Jacobs   16       U17
3    Bram Wouters   17       U19
4  Emma Verhoeven   15       U17
```

Five edges make four bins: from 10 to 12, from 12 to 14, and so on. A bin includes its *right* edge but not its left edge: an athlete of 12 falls in U13 (10 up to and including 12), an athlete of 13 in U15. So there is always one label fewer than there are edges.

2.3 Grouping on several columns and agg

You can group on two columns at once by passing a list. With `.agg(...)` you compute several statistics in one go and give each new column a name yourself:

```
athletes_with_age_group.groupby(["position", "age_group"], observed=True).agg(
    n_athletes=("athlete_id", "size"),
    mean_height_cm=("height_cm", "mean"),
)
```

output

```
                     n_athletes  mean_height_cm
position   age_group
Defender   U13                2      145.400000
           U15                5      158.250000
           U17                3      169.900000
```

output

```
           U19                2      174.150000
Forward    U13                7      153.350000
           U15                4      165.475000
...                         ...             ...
Goalkeeper U15                1      157.000000
           U17                3      174.466667
Midfielder U13                2      157.950000
           U15                5      164.440000
           U17                1      163.300000
           U19                4      171.475000
```

Read `mean_height_cm=("height_cm", "mean")` as: "make a column `mean_height_cm` by applying the function `mean` to the column `height_cm`". A few extra things:

- The index now has two levels (position and age group): a *MultiIndex*. `.reset_index()` turns them back into ordinary columns.
- `age_group` is a *categorical* column (made by `pd.cut`). `observed=True` makes sure combinations without athletes (e.g. goalkeepers in U13) don't show up as an empty row.

Try it yourself

Compute the maximum height per position. Then group by gender and count with `.size()` how many boys and girls there are.

3 Combining tables with `merge`

3.1 A key in two tables

`athletes` knows the position of every athlete, `tests` knows the sprint times. To look at sprint time per position, you have to combine the two. That works because both tables have an `athlete_id` column: the **key**.

A small example makes clear what happens:

```
people = pd.DataFrame({"athlete_id": ["A1", "A2", "A3"],
                       "name": ["Yana", "Stan", "Emma"]})
results = pd.DataFrame({"athlete_id": ["A1", "A1", "A2", "A9"],
                        "sprint_s": [5.3, 5.2, 5.9, 6.1]})
people.merge(results, on="athlete_id", how="inner")
```

output

```
  athlete_id  name  sprint_s
0         A1  Yana       5.3
1         A1  Yana       5.2
2         A2  Stan       5.9
```

For every row in `people`, pandas looks for the rows in `results` with the same `athlete_id`. Yana has two results, so she appears twice. Emma (A3) has no result and A9 has no name: with `how="inner"` both disappear.

```
people.merge(results, on="athlete_id", how="left")
```

output

```
  athlete_id  name  sprint_s
0         A1  Yana       5.3
1         A1  Yana       5.2
```

output
```
2         A2  Stan       5.9
3         A3  Emma       NaN
```

With `how="left"`, *every* row of the left table is kept. Emma is there, with `NaN` as her sprint time.

The kinds of joins

inner: only keys that are in both tables. **left**: all rows of the left table, filled in where possible. **right**: the same, from the right table. **outer**: everything from both tables. Check the number of rows after every merge: it tells you whether something disappeared or doubled.

With the real data:

```
merged_inner = athletes.merge(tests, on="athlete_id", how="inner")
merged_left = athletes.merge(tests, on="athlete_id", how="left")
(len(athletes), len(tests), len(merged_inner), len(merged_left))
```

output
```
(43, 114, 114, 114)
```

Here both have the same length, because every athlete has at least one test result. With real data that is rarely guaranteed.

3.2 The duplicated athlete

One athlete appears twice in `athletes`, with two different IDs. You find those rows with `duplicated`; with `keep=False` *all* copies are marked, not only the second one:

```
same_person = athletes.duplicated(subset=["name", "birthdate"], keep=False)
athletes.loc[same_person, ["athlete_id", "name", "birthdate"]]
```

output
```
   athlete_id       name   birthdate
10       A011  Liam Maes  2014-10-20
42       A043  Liam Maes  2014-10-20
```

The merge does exactly what it was asked, but does not know these are the same person. After the merge, that athlete's tests appear under two IDs:

```
dupe_name = athletes.loc[same_person, "name"].iloc[0]
merged_inner.loc[merged_inner["name"] == dupe_name,
                 ["athlete_id", "name", "test_moment", "sprint_30m_s"]]
```

output
```
    athlete_id       name  test_moment  sprint_30m_s
24        A011  Liam Maes            1          5.35
25        A011  Liam Maes            2          5.29
26        A011  Liam Maes            3          5.37
111       A043  Liam Maes            1          4.64
112       A043  Liam Maes            2          4.59
113       A043  Liam Maes            3          4.46
```

Every statistic you compute after this merge counts this athlete twice. The solution: remove the duplicates *before* the merge, by name and birthdate. After the merge that no longer works, because then the different test moments of one athlete also share the same name and birthdate: you would throw away real test results.

```
athletes_unique = athletes.drop_duplicates(subset=["name", "birthdate"],
↪  keep="first")
```

```
combined = athletes_unique.merge(tests, on="athlete_id", how="inner")
combined.shape
```

output

```
(111, 15)
```

3.3 Sets and `isin`

To find out which rows disappeared when removing the duplicate, you need two tools.

A **set** is a collection without duplicates and without order. The difference of two sets (`a - b`) gives the elements that are in `a` but not in `b`:

```
set(["A1", "A2", "A3"]) - set(["A2"])
```

output

```
{'A3', 'A1'}
```

`.isin(...)` gives a mask that is `True` for every row whose value is in the given list or set:

```
results["athlete_id"].isin({"A1", "A9"})
```

output

```
0     True
1     True
2    False
3     True
Name: athlete_id, dtype: bool
```

With these two building blocks (and `.sum()` on a mask) you can solve exercise 2 in the notebook.

4 Long and wide: `pivot_table` and `melt`

4.1 Two shapes for the same data

`combined` is **long**: one row per athlete *per test moment*. An athlete with three tests appears three times. The same information can also be **wide**: one row per athlete, with a column per test moment.

```
sprint_wide = combined.pivot_table(
    index="athlete_id", columns="test_moment", values="sprint_30m_s"
)
sprint_wide.head()
```

output

```
test_moment     1     2     3
athlete_id
A001         5.34  5.32   NaN
A002         5.97  5.74   NaN
A003         5.86  5.87   NaN
A004         5.33  5.25  5.28
A005         5.59  5.66   NaN
```

Read the three arguments like this:

- `index`: what becomes a row (one per athlete);
- `columns`: which values become columns (test moments 1, 2 and 3);
- `values`: which value goes in the cells (the sprint time).

If a combination is missing, you get `NaN`. If there are several rows for the same combination, `pivot_table` takes the mean by default.

4.2 Why wide is handy

In the wide shape, progress between two test moments is a single subtraction. Note: the column names here are *numbers*, not text. So you write `sprint_wide[2]`, not `sprint_wide["2"]`.

```
sprint_progress_1_to_2 = sprint_wide[2] - sprint_wide[1]
sprint_progress_1_to_2.describe()
```

output

```
count    41.000000
mean     -0.069512
std       0.110339
min      -0.270000
25%      -0.170000
50%      -0.070000
75%       0.010000
max       0.140000
dtype: float64
```

A negative number means progress here: a lower sprint time is faster. For jump height it is the other way round: higher is better.

4.3 Back to long: `melt`

`melt` does the opposite of `pivot_table`: columns become rows again. Plotly express (lab 5) prefers long data. First, `.reset_index()` turns the index back into an ordinary column:

```
sprint_long_again = sprint_wide.reset_index().melt(
    id_vars="athlete_id", var_name="test_moment", value_name="sprint_30m_s"
)
sprint_long_again.head()
```

output

	athlete_id	test_moment	sprint_30m_s
0	A001	1	5.34
1	A002	1	5.97
2	A003	1	5.86
3	A004	1	5.33
4	A005	1	5.59

`id_vars` are the columns that stay, `var_name` is the name of the new column holding the former column names, `value_name` the name of the column holding the values.

5 Time series: heart rate during a training session

`hr_session.csv` contains the heart rate and speed of five athletes per second during one training session. A table in which every row is a moment in time is called a **time series**.

5.1 Making a timestamp

The file has a date and the number of seconds since the start (`time_s`). Together they make a timestamp. `pd.to_timedelta(..., unit="s")` turns a number of seconds into a duration, which you can add to a date:

```
hr_session = pd.read_csv(f"{DATA_DIR}/hr_session.csv")
hr_session["timestamp"] = pd.to_datetime(hr_session["date"]) + pd.to_timedelta(
    hr_session["time_s"], unit="s"
)
hr_session.head(3)
```

output

	athlete_id	date	time_s	heart_rate_bpm	speed_kmh	timestamp
0	A001	2026-11-09	0	85.0	0.94	2026-11-09 00:00:00
1	A001	2026-11-09	1	80.0	2.84	2026-11-09 00:00:01
2	A001	2026-11-09	2	80.0	2.58	2026-11-09 00:00:02

5.2 Heart-rate zones

Coaches express intensity as a percentage of the *maximum* heart rate. A common rule of thumb estimates it as 220 minus the age. The age is in `athletes`, so first a merge:

```
ages = athletes[["athlete_id", "age"]]
hr_with_age = hr_session.merge(ages, on="athlete_id", how="left")
hr_with_age["hr_max"] = 220 - hr_with_age["age"]
hr_with_age["pct_hr_max"] = (
    100 * hr_with_age["heart_rate_bpm"] / hr_with_age["hr_max"]
)
hr_with_age[["athlete_id", "heart_rate_bpm", "hr_max", "pct_hr_max"]].head(3)
```

output

	athlete_id	heart_rate_bpm	hr_max	pct_hr_max
0	A001	85.0	204	41.666667
1	A001	80.0	204	39.215686
2	A001	80.0	204	39.215686

`pd.cut` divides the percentage into five zones. The odd number 1000 as the last edge makes sure a percentage above 100 still falls in zone 5:

```
hr_zoned = hr_with_age.assign(
    hr_zone=lambda d: pd.cut(
        d["pct_hr_max"],
        bins=[0, 60, 70, 80, 90, 1000],
        labels=["Zone 1", "Zone 2", "Zone 3", "Zone 4", "Zone 5"],
    )
)
hr_zoned["hr_zone"].value_counts()
```

output

```
hr_zone
Zone 4    6172
Zone 1    2344
Zone 3    1517
Zone 5    1369
Zone 2     573
Name: count, dtype: int64
```

5.3 Time per zone

Every row is one second. So the number of rows per athlete and per zone, divided by 60, gives the number of minutes:

```
time_per_zone_minutes = (
    hr_zoned.groupby(["athlete_id", "hr_zone"], observed=True).size() / 60
).rename("minutes").reset_index()
time_per_zone_minutes.head(5)
```

output

```
  athlete_id hr_zone    minutes
0       A001  Zone 1   8.350000
1       A001  Zone 2   1.583333
2       A001  Zone 3   7.883333
3       A001  Zone 4  19.400000
4       A001  Zone 5   2.683333
```

If you only want certain zones, first filter the rows with `.isin([...])` and then group. That is the core of exercise 4.

5.4 Rolling average and resampling

A heart rate per second jumps up and down. Two techniques make the series calmer. Both need a time series *sorted* by time, with the time as the index (`set_index("timestamp")`):

```
one_athlete = hr_zoned.loc[hr_zoned["athlete_id"] == "A001"]
one_athlete = one_athlete.sort_values("timestamp")
hr_series = one_athlete.set_index("timestamp")["heart_rate_bpm"]
```

A **rolling average** (`rolling`) computes, at every moment, the mean of the past 30 seconds. You keep as many rows, but the line gets smoother:

```
hr_series.rolling("30s").mean().iloc[[0, 1, 29, 30, 31]]
```

output

```
timestamp
2026-11-09 00:00:00    85.000000
2026-11-09 00:00:01    82.500000
2026-11-09 00:00:29    81.833333
2026-11-09 00:00:30    81.800000
2026-11-09 00:00:31    81.933333
Name: heart_rate_bpm, dtype: float64
```

Resampling (`resample`) groups the time series into fixed blocks, here of one minute, and summarizes each block. You get far fewer rows: one per minute.

```
hr_series.resample("1min").mean().head(4)
```

output

```
timestamp
2026-11-09 00:00:00     82.150000
2026-11-09 00:01:00     85.000000
2026-11-09 00:02:00     99.933333
2026-11-09 00:03:00    110.466667
Freq: min
Name: heart_rate_bpm, dtype: float64
```

`rolling` or `resample`?

`rolling` gives one value per original row (a sliding window); useful to make a chart smoother. `resample` gives one value per time block; useful to summarize per minute or per quarter of an hour. You can think of `resample` as a `groupby` on time.

6 The mini-case: a training report

In the mini-case (in pairs) you build one row per athlete with the minutes in zone 4–5, the maximum speed and the distance covered. Two ideas help you.

Distance from speed. Speed is in km/h and there is one measurement per second. So in one second you cover `speed_kmh / 3600` kilometres. The sum of those distances over the whole session is the total distance.

Combining Series via the index. When two Series have the same index (e.g. the `athlete_id`), pandas puts the values next to the right row in an assignment or calculation, even if the order is different. This is called *aligning on the index*:

```
report = pd.DataFrame({"max_speed": [27.1, 24.5]}, index=["A002", "A001"])
extra = pd.Series([10.0, 12.5], index=["A001", "A002"])
report["minutes"] = extra
report
```

output

```
      max_speed  minutes
A002       27.1     12.5
A001       24.5     10.0
```

A001 gets 10.0 and A002 gets 12.5, even though they were in a different order in `extra` than in `report`.

7 Summary

Code	What it does
`df.groupby("a")["b"].mean()`	mean of `b` per value of `a`
`df.groupby("a").size()`	number of rows per group
`df.groupby(["a", "b"]).agg(name=("col", "mean"))`	several named statistics per group
`pd.cut(col, bins=..., labels=...)`	divide numbers into categories
`.reset_index()`	turn the index back into ordinary columns
`left.merge(right, on="key", how="inner")`	combine two tables on a key
`set(a) - set(b)`, `col.isin(list)`	difference of sets, “is in” mask
`df.pivot_table(index=, columns=, values=)`	from long to wide
`df.melt(id_vars=, var_name=, value_name=)`	from wide to long
`pd.to_timedelta(col, unit="s")`	number of seconds to a duration
`s.rolling("30s").mean()`	rolling average over 30 seconds
`s.resample("1min").mean()`	mean per block of one minute

Common mistakes

- Not checking the number of rows after a merge.
- Removing duplicates *after* the merge instead of before.
- Using `.count()` to count rows when there are missing values; use `.size()`.
- Writing column names as text after a `pivot_table` (`wide["2"]`) when they are numbers (`wide[2]`).
- Using `rolling` or `resample` on data that is not sorted by time, or without a time index.

Glossary

groupby	split into groups, compute something per group, and combine the results.
aggregation	summarizing a group of values into one value (mean, sum, count).
MultiIndex	index with several levels, e.g. position and age group.
key	column used to combine two tables.
join	way of combining: inner, left, right or outer.
long / wide	one row per measurement, or one row per athlete with a column per measurement.
time series	table in which every row is a moment in time.